

Technical Design & Development Report — UPAO-MAS-EDU

SECCIÓN 0 — PORTADA Y METADATOS DEL PROYECTO

Campo	Descripción
Nombre del proyecto	PLATAFORMA MULTIAGENTE PEDAGÓGICA OBSERVABLE CON ORQUESTACIÓN ADAPTATIVA, RETRIEVAL CONTEXTUAL Y BENCHMARKING REPRODUCIBLE PARA LA ADAPTACIÓN DE CONTENIDO EDUCATIVO EN FUNDAMENTOS DE PROGRAMACIÓN
Nombre de la plataforma desarrollada	UPAO-MAS-EDU
Tipo de solución	Solución híbrida de software compuesta por una aplicación web con arquitectura multiagente basada en Inteligencia Artificial Generativa para la personalización adaptativa del proceso de aprendizaje.
Dominio de aplicación	Educación Superior – Sistemas Inteligentes para el Aprendizaje Adaptativo en Fundamentos de la Programación.
Palabras clave	Multi-Agent Systems; Adaptive Learning Systems; Large Language Models (LLMs); Intelligent Tutoring Systems; Swarm Intelligence; Artificial Intelligence in Education (AIED)
Repositorio del código	https://github.com/Renato5Lara/multiagent-llm-education (público)
Dataset	UPAO-MAS-EDU utiliza un conjunto de datasets pedagógicos propios integrados dentro del repositorio del proyecto, destinados al soporte, configuración y evaluación del comportamiento de los agentes especializados responsables de la adaptación del contenido educativo. Actualmente estos conjuntos de datos forman parte del código fuente y no cuentan con un repositorio independiente ni con un identificador DOI.

SECCIÓN 1 — PROBLEMA Y MOTIVACIÓN TÉCNICA

1.1 Descripción del problema real

La desaprobación y el abandono en cursos introductorios de programación no es un fenómeno anecdótico: Watson y Li (2014), tras analizar 161 cursos de programación en 51 instituciones de 15 países, reportan una tasa promedio de fracaso del 32.3% (cifra citada en Rahmani et al., 2024). Bennedsen y Caspersen (2019) confirman que esta cifra es estructural y no puntual, situándola de forma sostenida entre el 28% y el 33% (también vía Rahmani et al., 2024). La propia revisión sistemática de Rahmani et al. (2024) sobre deserción en educación superior en línea muestra, además, que los programas impartidos íntegramente en línea presentan tasas de abandono significativamente más altas que sus equivalentes presenciales.

La causa raíz identificada en la literatura no es la dificultad intrínseca del contenido, sino el diseño instruccional. Rahmani et al. (2024) señalan el diseño y la estructura del curso —no la capacidad del estudiante— como el principal factor negativo asociado al abandono. Beauchemin et al. (2024) lo atribuyen específicamente al enfoque estandarizado de "talla única" (one-size-fits-all), que ignora las diferencias individuales de ritmo y capacidad y provoca una caída documentada del compromiso (engagement) y la motivación, ambos precursores directos del abandono. Smaili et al. (2023) llegan a una conclusión convergente desde otro ángulo: las plataformas educativas fracasan en retener estudiantes porque no abordan su heterogeneidad real (antecedentes, nivel de conocimiento previo, hábitos de aprendizaje).

Las soluciones actuales — LMS tradicionales, plataformas de ejercicios con retroalimentación estática, o tutores basados en un único agente de lenguaje — heredan esta misma limitación estructural: entregan contenido homogéneo y no dejan trazabilidad verificable de por qué se tomó una decisión pedagógica determinada para un estudiante concreto.

1.2 Brecha tecnológica identificada

La brecha técnica específica que aborda UPAO-MAS-EDU no es la ausencia de inteligencia artificial en la educación —existen múltiples tutores basados en LLM de agente único—, sino la ausencia de un mecanismo de **decisión pedagógica distribuida y auditable**: un proceso en el que múltiples agentes especializados (investigación de contenido, generación de código, revisión de calidad) deliberen, alcancen consenso ponderado y dejen memoria compartida y explicabilidad de cada adaptación, en lugar de que un único modelo genere contenido de forma opaca.

Esta brecha justifica una solución computacional —no meramente pedagógica— porque requiere: orquestación de múltiples procesos de inferencia concurrentes, un mecanismo determinista de consenso entre agentes, persistencia de estado y memoria narrativa entre sesiones, e instrumentación de observabilidad capaz de reconstruir la cadena causal de cada decisión.

1.3 Pregunta de investigación técnica

¿En qué medida una arquitectura de orquestación multiagente basada en inteligencia de enjambre permite adaptar el contenido educativo al perfil individual del estudiante, en el contexto del curso Fundamentos de la Programación, con trazabilidad completa del proceso de decisión?

1.4 Objetivo general y objetivos específicos

Objetivo general: Diseñar, implementar y validar una plataforma de orquestación multiagente basada en inteligencia de enjambre, capaz de adaptar el contenido educativo de Fundamentos de la Programación al perfil individual del estudiante, con trazabilidad y explicabilidad completas del proceso de decisión.

Objetivos específicos:

1. Diseñar una arquitectura multiagente con roles especializados capaz de producir contenido educativo adaptado mediante consenso ponderado (*Sección 3*).
2. Implementar un mecanismo de memoria compartida e inferencia colectiva que preserve la continuidad narrativa y pedagógica entre módulos consecutivos del aprendizaje (*Sección 4*).
3. Instrumentar el sistema con observabilidad y detección de anomalías de enjambre que permitan auditar cada decisión adaptativa (*Sección 4*).

4. Establecer un framework de benchmarking reproducible que mida la precisión pedagógica, la diversidad de fuentes de contenido, la calidad del código generado y la tasa de revisión del sistema multiagente (*Sección 5*).
5. Evaluar si la arquitectura multiagente propuesta produce una adaptación de contenido más rica y explicable que un enfoque de agente único, respondiendo la pregunta de investigación técnica (*Sección 6*).

1.5 Alcance y limitaciones declaradas

UPAO-MAS-EDU se limita, de manera deliberada, a un único curso — Fundamentos de la Programación, con sus nueve módulos (Introducción, Variables y Tipos de Datos, Operadores y Expresiones, Condicionales, Bucles, Funciones, Arreglos, Recursividad y Programación Orientada a Objetos básica) — y a tres roles de usuario (estudiante, docente, administrador). Quedan explícitamente fuera de alcance: la gestión curricular institucional, el soporte multi-asignatura, la administración académica avanzada y cualquier funcionalidad de red social o colaboración en tiempo real ajena a la hipótesis de adaptación multiagente.

Como restricciones declaradas: el sistema opera con datasets pedagógicos propios de tamaño acotado, depende de proveedores de LLM externos (OpenAI, con degradación a plantillas locales si no están disponibles) y su validación experimental —como se detalla con honestidad metodológica en la Sección 5— se realiza sobre escenarios controlados, no sobre una cohorte real de estudiantes en producción a escala.

1.6 Contribución técnica principal

La contribución técnica de UPAO-MAS-EDU no es la existencia aislada de sus componentes sino su **combinación en una única arquitectura operativa y observable**: un sistema que adapta contenido multimodal en tiempo real mediante deliberación distribuida entre agentes, conserva memoria narrativa entre sesiones, expone explicabilidad verificable de cada decisión y se evalúa mediante un framework de benchmarking reproducible propio, aplicado específicamente al dominio de enseñanza de programación introductoria.

SECCIÓN 2 — REVISIÓN DE LITERATURA TÉCNICA

2.1 Marco conceptual técnico

Sistemas Multiagente (MAS). Un MAS coordina varios agentes con roles y capacidades diferenciadas, cada uno responsable de una parte del problema, en lugar de resolverlo con un único proceso de inferencia. El fundamento técnico es la separación de responsabilidades: un agente que investiga contenido no debe ser el mismo que lo valida, porque el valor del diseño está en el contraste entre perspectivas, no en la suma de su capacidad individual. En UPAO-MAS-EDU esto se traduce directamente en la separación entre el Agente de Investigación (**ResearchAgent**), el Agente de Generación de Código (**ProgrammerAgent**) y el Agente de Revisión (**ReviewerAgent**): cada uno opera con un objetivo distinto y su desacuerdo es una señal de diseño, no un error a eliminar. *(Nota de consistencia: una cuarta clase, `VisualDesignerAgent`, existe físicamente en el código pero no está instanciada por ningún flujo en vivo del sistema — es código huérfano, no un agente operativo, y por eso queda fuera de esta enumeración.)* Esta separación de roles es la base sobre la que se apoya el siguiente concepto: sin múltiples agentes independientes, no existe nada que poner en consenso.

Inteligencia de Enjambre. Los algoritmos de enjambre (ACO, PSO, ABC) resuelven problemas de búsqueda sobre espacios demasiado grandes para una regla determinista, mediante agentes simples que convergen colectivamente hacia una solución razonable, no necesariamente óptima. El fundamento no es la sofisticación individual del agente, sino la explotación de la interacción entre muchos agentes simples. En UPAO-MAS-EDU este principio no se usa para optimizar una única métrica numérica (como en la mayoría de los trabajos revisados en 2.2), sino como mecanismo de coordinación entre los agentes generativos del MAS: el Coordinador de Enjambre decide, mediante deliberación y voto ponderado, qué propuesta de adaptación pedagógica prevalece. Esto conecta directamente con el siguiente concepto, porque una decisión de enjambre sin justificación pedagógica no tiene valor educativo por sí sola.

Aprendizaje Adaptativo y Personalizado. Este concepto cubre el ajuste dinámico de contenido, ritmo o modalidad según un perfil inferido del estudiante. Su fundamento técnico es el modelado del estudiante como un estado observable que condiciona la siguiente decisión de contenido, no un dato estático fijado al inicio del curso. En UPAO-MAS-EDU esto es el objetivo funcional que el MAS y el enjambre existen para servir: el perfil de aprendizaje del estudiante es la entrada que el Coordinador de Enjambre pondera al decidir entre propuestas de los agentes. Esto lleva al cuarto concepto: ¿con qué genera el sistema el contenido que se adapta?

LLM aplicados a Tutoría y Programación. El uso de modelos de lenguaje grande como generadores de contenido pedagógico o agentes conversacionales de andamiaje, en lugar de bancos de contenido estático. El fundamento es que un LLM puede producir explicaciones, ejercicios y retroalimentación condicionados al contexto específico del estudiante, a un costo de trazabilidad y confiabilidad que debe mitigarse explícitamente. En UPAO-MAS-EDU, los agentes de Investigación, Generación de Código y Revisión están implementados sobre un LLM; el Agente de Revisión (**ReviewerAgent**) existe específicamente porque un LLM generador no es, por diseño, su propio validador confiable.

2.2 Estado del arte de soluciones similares

Ref	Año	Tipo de solución	Técnica/Tecnología	Dataset/Contexto	Métrica principal	Limitación reportada	Contribución presente t
López-Goyez et al.	2026	MAS autoadaptativo (ELA Tutor)	RL Meta-Agent + LLM sobre Moodle (n8n/Django)	Casos reales y simulados, educación superior	Mejora en eficiencia y selección de agentes (Mann-Whitney, Kruskal-Wallis)	Requiere expandir supervisión docente (HITL)	Respalda de un me coordinac agentes y qué la sup docente q prevista e document arquitecto descartad
Mohamedhen et al.	2024	MAS de recomendación de objetos de aprendizaje	MAS + CNN + MLP, modelo Felder-Silverman	100 estudiantes, 10 cursos, IEEE LOM	Accuracy, Precision, Recall, F1	Necesita más estrategias de ML	Valida el modelo d aprendiza entrada d consume

Ref	Año	Tipo de solución	Técnica/Tecnología	Dataset/Contexto	Métrica principal	Limitación reportada	Contribución presente trabajo
							Coordinación de Enjambre
Zhao et al.	2024	Planificación dinámica de rutas (LPP)	ACO + K-means + DTW	300 estudiantes, curso de Semiótica	Precisión 96.6%, finalización 90%	Métodos tradicionales de LPP no se adaptan a complejidad dinámica	Fundamento de optimización colectiva para secuencias de actividades, antecedente del Coordinador de Enjambre
Imamah et al.	2024	Rutas personalizadas (ACOIRT)	ACO + Teoría de Respuesta al Ítem	80 estudiantes, Estructuras de Datos	Mejora post-test 60.8–127.8% (p=0.002)	Iteraciones repetitivas pueden aburrir	Evidencia de que la secuencia de actividades en un enjambre puede mejorar los resultados, no solo en entornos computacionales
Al-Ahmad et al.	2022	Predicción temprana de rendimiento en equipos	PSO + K-Nearest Neighbors	Dataset SETAP	AUC 96%, F-Measure 93.9%	Difícil generalizar entre datasets	Antecedente para detectar problemas tempranos de rendimiento relevante para diagnóstico de estudiantes
Sheng et al.	2023	Secuenciación curricular adaptativa (ACS)	Group-Theoretic PSO	OULAD, 50–1000 materiales	Fitness score 9.266–16.214	No garantiza óptimo global; alto costo computacional	Advierte sobre el costo de explorar espacios de búsqueda grandes (información de Coordinador de Enjambre en un espacio de decisión a módulos)
Mahawar et al.	2025	Predicción de éxito académico	ACO + ABC + SMOTE + Árbol de Decisión	Minería de datos educativos	Accuracy 98.15%, F1 98.10%	Combinar optimizadores genera problemas de escalabilidad	Justifica el uso de un único mecanismo de consenso en un enjambre para apilar varias optimizaciones
Yilmazer y Özel	2024	Recomendación diversa (AcoRec)	ACO continuo + bayesiano + Adam	MovieLens, Pinterest, Netflix	NDCG@10/20, Recall@10, Coverage	Un modelo puede sobresalir en un dataset y fallar en otro	Aporta evidencia de trade-off entre relevancia y diversidad relevante del voto personal del consejero
Li y Lu	2025	Recomendación multimodal	Autoencoders + RBM + grafos + MLP	480 registros, 2 semestres	MAE 0.01, MSE 0.0053	Dataset reducido limita generalización	Respaldar la adaptación multimodal para un objetivo de recomendación con limitaciones de recursos paralelos en datasets pequeños del proyecto

Ref	Año	Tipo de solución	Técnica/Tecnología	Dataset/Contexto	Métrica principal	Limitación reportada	Contribución presente t
Zhu et al.	2024	Microaprendizaje adaptativo (AML)	Modelo logístico de tres parámetros	Adultos en contexto laboral	Reducción de carga cognitiva ($p<0.05$)	Amenazas a validez interna/externa	Fundame cognitiva variable c monitoreo adaptació contenido
Naseer et al.	2024	Rutas de aprendizaje personalizadas	Deep Learning + analítica predictiva	4 cursos, 240 estudiantes	Mejora 25% ($p=0.00045$)	Fallos de integración curricular, UX poco intuitiva	Evidencia adaptació resultados advier UX que i diseño de experienc estudiante
Huang et al.	2025	Tutoría inteligente en programación (SP-TeachLLM)	Agentes LLM (GPT-4o, LLaMA-3) + CRAG + RL	Benchmarks HumanEval, MBPP	Incremento en precisión de código (CGA)	Estudiantes simulados restringen validez externa	Justifica el uso de y MBPP benchmark Agente de Generació Código (Programa
Vaccaro et al.	2025	Personalización de textos STEM	2 agentes LLM (Perfilador/Reescritor)	RCT piloto, 23 estudiantes	Mejora percibida en personalización	No mide aprendizaje a largo plazo	Evidencia viabilidad LLM espe antecedent de los ag Investigac Revisión
Yan et al.	2025	Programación colaborativa con LLM	LLM (GPT-4) como compañero de aprendizaje	5 semanas, 82 estudiantes K-12	Reducción de carga cognitiva	Lenguaje complejo (C++) redujo autoeficacia	Advierte riesgo de amplio, re decisión proyecto Fundame Programa
Pardos y Bhandari	2024	Ayudas automáticas matemáticas	LLM (ChatGPT 3.5) + auto-consistencia	274 participantes, Mechanical Turk	Ganancia de aprendizaje 17%	Alta tasa de abandono (30%), modelo cerrado	Respald consisten mitigació alucinaci a la fiabil Agente de (Review

2.3 Análisis comparativo de gaps

Característica	López-Goyez et al. (2026)	Zhao et al. (2024)	Huang et al. (2025)	Vaccaro et al. (2025)	UPAO-MAS-EDU
Arquitectura multiagente	✓	✗	✓	✓	✓
Consenso ponderado entre agentes	✗	✗	✗	✗	✓

Característica	López-Goyez et al. (2026)	Zhao et al. (2024)	Huang et al. (2025)	Vaccaro et al. (2025)	UPAO-MAS-EDU
Memoria compartida entre sesiones	X	X	X	X	✓
Observabilidad/trazabilidad de la decisión	X	X	X	X	✓
Benchmarking reproducible	X	X	✓	X	✓

Dentro del corpus bibliográfico analizado en este estudio, no se identificó un trabajo que integrara simultáneamente estas cinco capacidades. Cada grupo temático resuelve una parte del problema de forma aislada. Desde una perspectiva de ingeniería de software, esto implica que la plataforma debe ser capaz de sostener las cinco capacidades a la vez sin heredar ninguna solución completa de la literatura.

2.4 Justificación de la elección tecnológica

Alternativa	Ventajas	Desventajas	Motivo de descarte/elección
Agente único de LLM	Simplicidad de implementación, bajo costo de orquestación	Sin deliberación ni contraste de perspectivas; el generador es su propio validador	Descartado: no permite auditar por qué se tomó una decisión pedagógica
RAG clásico (recuperación + generación)	Reduce alucinación con contexto recuperado, bajo costo	Sigue siendo un único punto de decisión; sin coordinación entre roles	Descartado como arquitectura principal; se conserva como técnica dentro de agentes individuales
Workflow secuencial fijo (pipeline sin deliberación)	Fácil de implementar y depurar	Rígido: no hay lugar para desacuerdo entre etapas ni adaptación dinámica	Descartado: no sostiene consenso ponderado ni memoria compartida entre etapas
Arquitectura multiagente con consenso (enjambre)	Permite roles especializados, consenso auditable, y coordinación adaptativa	Mayor complejidad de orquestación y de prueba	Elegida: es la alternativa seleccionada porque satisface simultáneamente las cinco capacidades identificadas en el análisis comparativo (tabla 2.3)

Transición hacia el diseño de la solución. El análisis de la literatura evidencia que las soluciones existentes resuelven parcialmente el problema mediante enfoques independientes. En consecuencia, la arquitectura de UPAO-MAS-EDU se diseña integrando estas capacidades en una única plataforma, priorizando la coordinación entre agentes, la trazabilidad de las decisiones, la persistencia del estado y la observabilidad del proceso adaptativo. La siguiente sección describe cómo estos principios se materializan en la arquitectura propuesta.

SECCIÓN 3 — DISEÑO DE LA SOLUCIÓN TECNOLÓGICA

3.1 Visión general de la arquitectura

Como parte de una estrategia de migración incremental, el repositorio mantiene tres mecanismos de orquestación con responsabilidades diferenciadas, en transición hacia la arquitectura basada en LangGraph:

Subsistema	Estado	Función	Acceso
Runtime LangGraph (RFC-0004)	Vigente	Adaptación integral del estudiante	FastAPI → <code>/api/runtime</code> (Boundary, integración parcial)
Orquestación de Servicios (app/services/)	Vigente	Generación de contenido de módulo y planificación semanal	FastAPI → <code>/api/students</code> , <code>/api/pedagogy</code>
BaseAgent (app/agents/)	Legado / experimental	Grupo de control del benchmark Legacy vs. Runtime	Sin ruta HTTP registrada en producción

1. Runtime LangGraph (`backend/runtime/`). Implementa la visión de cuatro capas de RFC-0001 (Domain / Kernel / Graph Engine / Platform Boundary — el control de la ejecución reside en el grafo, no en el LLM):

- **Domain** (runtime/domain/): ocho capacidades independientes — `adaptar`, `diagnosticar`, `evaluar`, `modelar`, `orientar`, `remediar`, `tutorizar`, `validar`.
- **Kernel** (runtime/kernel/): `reducers` (RFC-0003, `LearningState`), memoria (RFC-0005), deliberación y consenso (RFC-0006), eventos y transiciones.
- **Graph Engine** (runtime/engine/): `runtime/engine/graph/walkthrough.py` es el único módulo del repositorio que instancia `langgraph.graph.StateGraph`.
- **Platform Boundary** (runtime/boundary/, RFC-0010): ya cableado a FastAPI vía `backend/app/api/routes/runtime.py` (prefijo `/api/runtime`) y `backend/app/services/runtime_bridge.py`, con una conexión a PostgreSQL propia y aislada (`runtime_connection.py`, ADR-0009).

Brecha declarada explícitamente: esta integración es *funcionalmente completa para el recorrido del estudiante*, pero no para el docente — el roadmap interno del proyecto sigue listando "Boundary completo" como plataforma operativa futura, posterior a "Runtime del Estudiante (COMPLETADO)".

2. Orquestación de Servicios (`app/services/`). Un segundo mecanismo, más simple, coordina agentes de clases planas (sin herencia de `BaseAgent`) para dos propósitos distintos:

- **Generación de contenido de módulo (estudiante):** `module_orchestration_service.py`, alcanzado desde `POST /api/students/module/{id}/orchestrate`, invoca únicamente al Agente de Investigación (`ResearchAgent`) — que realiza retrieval vía Tavily y su propia validación de consistencia — seguido de una llamada directa a `LLMService` (modelo `gpt-4o-mini`) para generar los bloques de concepto. Esta ruta **no** instancia `ProgrammerAgent` ni `ReviewerAgent`.
- **Planificación pedagógica semanal (docente):** `weekly_pedagogy_service.py` (`PedagogicalOrchestrationService`), alcanzado desde `POST /api/pedagogy/courses/{course_id}/weekly-plans`, instancia tanto el Agente de Investigación (`ResearchAgent`) como el Agente de Revisión (`ReviewerAgent`) — y este último, por defecto de su propio constructor, instancia también el Agente de Generación de Código (`ProgrammerAgent`) si no se le pasa uno explícitamente.

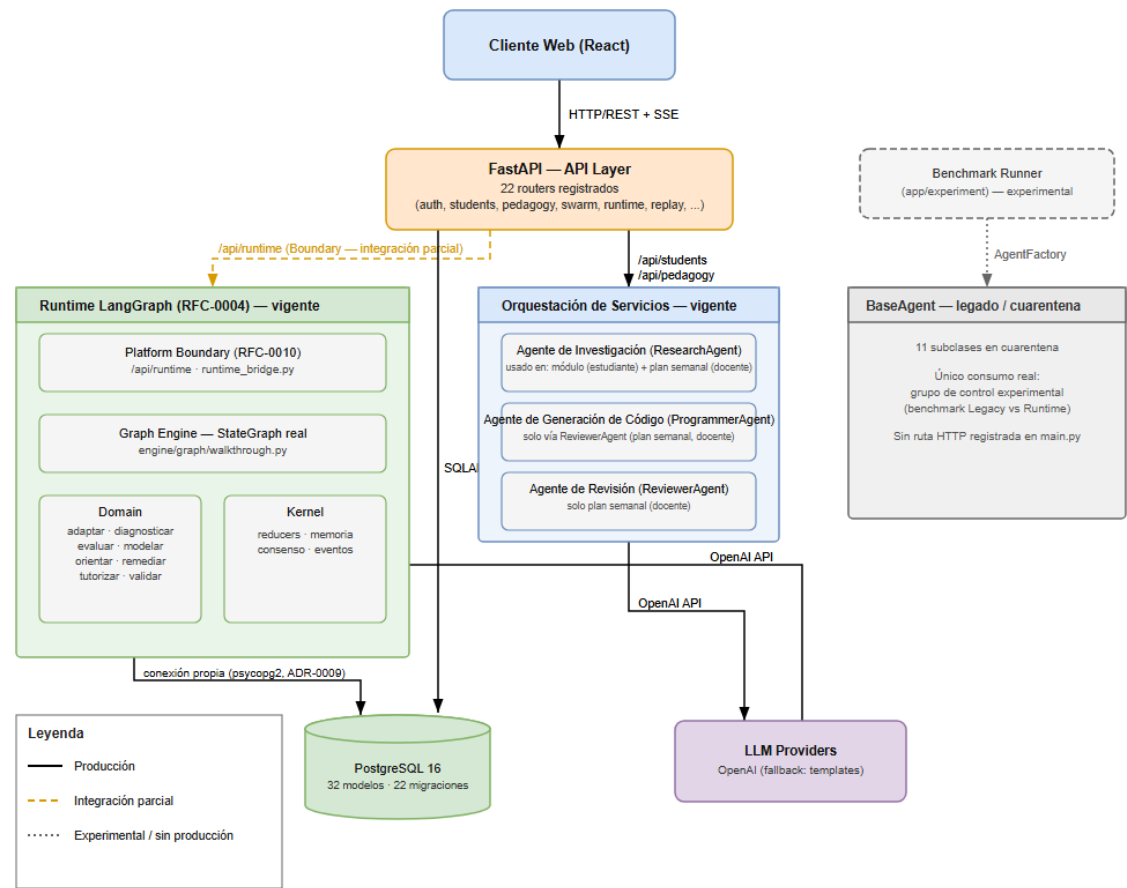
Ninguno de los dos flujos usa LangGraph ni pasa por el Boundary.

3. BaseAgent (`app/agents/`). Los archivos existen físicamente y once subclases siguen heredando de `BaseAgent`, en cuarentena declarada por su propio `__init__.py`. Su único consumo real es el grupo de control experimental del benchmark Legacy vs. Runtime (CONCEPT-0001); ningún router registrado en `main.py` lo alcanza.

Capa de persistencia (transversal): FastAPI expone 22 routers sobre PostgreSQL 16 vía SQLAlchemy async (32 modelos) y 22 migraciones Alembic con historial de ramas y merges reales.

Figura 1. Arquitectura general de UPAO-MAS-EDU. Arquitectura actual del sistema, mostrando los tres mecanismos de orquestación, sus responsabilidades y las relaciones entre la aplicación FastAPI, el Runtime LangGraph, la orquestación de servicios, el subsistema experimental BaseAgent y la capa de persistencia. (Archivo: `upao\_arquitectura\_fig1.svg`, adjunto.)

Figura 1 — Arquitectura de UPAO-MAS-EDU (estado real del repositorio)



La arquitectura presentada establece la organización estructural del sistema y delimita las responsabilidades de cada subsistema. Sobre esta base se definen, en las siguientes subsecciones, los requerimientos técnicos, los componentes funcionales y los mecanismos de interacción que materializan el comportamiento de la plataforma.

3.2 Especificación de requerimientos técnicos

3.2.1 Requerimientos funcionales

Gestión de usuarios

ID	Requerimiento	Prioridad	Vinculado a objetivo
RF-01	Autenticación y gestión de roles (estudiante/docente/admin)	P0	Transversal (infraestructura de acceso; no vinculado a un objetivo específico)

Trazabilidad complementaria:

ID	Fuente	Justificación técnica	Evidencia de implementación	Estado
RF-01	THESIS_SCOPE_FREEZE.md	Control de acceso diferenciado por actor	auth.router, users.router, modelo User/UserRole, LoginAttempt	Implementado

Personalización del aprendizaje

ID	Requerimiento	Prioridad	Vinculado a objetivo
RF-02	Diagnóstico inicial del estudiante	P1	Objetivo general (1.4)
RF-03	Generación de perfil de aprendizaje	P1	Objetivo general (1.4)
RF-04	Adaptación de contenido educativo al perfil del estudiante	P1	Objetivo general (1.4)
RF-07	Evaluación del estudiante	P1	Objetivo específico 5 (1.4)

Trazabilidad complementaria:

ID	Fuente	Justificación técnica	Evidencia de implementación	Estado
RF-02	THESIS_SCOPE_FREEZE.md	La personalización requiere un perfil real de entrada	Capacidad runtime/domain/diagnosticar, modelo DiagnosticResult, router knowledge_test	Implementado
RF-03	THESIS_SCOPE_FREEZE.md	Insumo directo del proceso de adaptación	Modelo StudentProfile, capacidad runtime/domain/modelar	Implementado
RF-04	THESIS_SCOPE_FREEZE.md + Objetivo general (1.4)	Núcleo de la hipótesis de investigación	Capacidad runtime/domain/adaptar, endpoint GET /api/students/adaptive-decision/{course_id} vía runtime_bridge.py	Implementación parcial — completo para el flujo del estudiante; extensión al flujo docente en roadmap
RF-07	THESIS_SCOPE_FREEZE.md	Cierre del ciclo diagnóstico → adaptación → evaluación	Capacidad runtime/domain/evaluar, modelo EvaluationAttempt	Implementado

Orquestación pedagógica

ID	Requerimiento	Prioridad	Vinculado a objetivo
RF-05	Generación de contenido de módulo mediante	P1	Objetivo específico 1 (1.4)

ID	Requerimiento	Prioridad	Vinculado a objetivo
	investigación asistida por IA		
RF-06	Planificación pedagógica semanal con contraste multiagente (investigación + revisión)	P1	Objetivo específico 1 (1.4)

Trazabilidad complementaria:

ID	Fuente	Justificación técnica	Evidencia de implementación	Estado
RF-05	THESIS_SCOPE_FREEZE.md (Sistema Multiagente)	Fundamentar el contenido generado en fuentes reales recuperadas, en vez de generación directa sin respaldo	Agente de Investigación (ResearchAgent, retrieval Tavily + validación de consistencia) + LLMService para bloques de concepto, vía module_orchestration_service.py, endpoint POST /api/students/module/{id}/orchestrate	Implementado
RF-06	THESIS_SCOPE_FREEZE.md (rol docente) + Objetivo específico 1 (1.4)	Contraste de perspectivas entre agentes en vez de generación de agente único (brecha de 2.3)	weekly_pedagogy_service.py (PedagogicalOrchestrationService) instancia Agente de Investigación y Agente de Revisión (que a su vez instancia el Agente de Generación de Código), vía POST /api/pedagogy/courses/{course_id}/weekly-plans	Implementado

Observabilidad e investigación

ID	Requerimiento	Prioridad	Vinculado a objetivo
RF-08	Explicabilidad de la adaptación	P2	Objetivo específico 5 (1.4)
RF-09	Observabilidad y replay cognitivo del proceso multiagente	P2	Objetivo específico 3 (1.4)
RF-10	Framework de benchmarking reproducible	P3	Objetivo específico 4 (1.4)

Trazabilidad complementaria:

ID	Fuente	Justificación técnica	Evidencia de implementación	Estado
RF-08	RFC-0007 + THESIS_SCOPE_FREEZE.md	La hipótesis exige trazabilidad de la	Modelo AgentDecisionTraceRecord (con índices compuestos por	Implementación parcial — la persistencia de la traza está

ID	Fuente	Justificación técnica	Evidencia de implementación	Estado
		decisión, no solo el resultado	correlación causal), router evidence	implementada; las técnicas específicas de explicabilidad (SHAP/LIME) no forman parte de esta versión
RF-09	RFC-0007 + THESIS_SCOPE_FREEZE.md	Ninguna solución revisada en la Sección 2 ofrece esta capacidad de forma nativa	Router replay , router swarm/swarm_demo , event_outbox.py	Implementado — con una limitación de seguridad pendiente: ninguno de los endpoints de `replay` declara autenticación (ver 3.6)
RF-10	THESIS_SCOPE_FREEZE.md + Objetivo específico 4 (1.4)	Valida el objetivo específico 4	Modelo ExperimentResult , router research , módulo backend/app/benchmark/	Implementación parcial (infraestructura existente y validada end-to-end; el modo de evaluación actual usa un evaluador determinista con semilla, no llamadas reales a LLM — ver Sección 5)

3.2.2 Requerimientos no funcionales

Categoría	Requisito	Implementación	Criterio verificable
Rendimiento	Procesamiento asíncrono de extremo a extremo	FastAPI async + SQLAlchemy async	Ninguna operación del runtime bloquea el hilo principal mediante llamadas síncronas en una ruta async
Disponibilidad	Persistencia transaccional con historial reversible	PostgreSQL 16 + Alembic (22 migraciones)	Toda migración define down_revision y es reversible sin pérdida de datos
Escalabilidad	Aislamiento de conexión entre runtime y aplicación	Conexión propia del runtime vía psycopg2 (ADR-0009)	El runtime nunca reutiliza ni compite por la sesión async de la aplicación
Observabilidad	Reconstrucción verificable de cada decisión adaptativa	Replay cognitivo, swarm_diagnostics , event_outbox	Toda decisión adaptativa es recuperable vía el endpoint de replay sin

Categoría	Requisito	Implementación	Criterio verificable
			intervención manual en la base de datos
Seguridad	Autenticación y autorización por rol	JWT (python-jose, HS256) + UserRole + LoginAttempt (bloqueo tras 3 intentos/5 min)	Ningún endpoint protegido acepta solicitudes sin un token JWT válido — excepción pendiente de corrección: el router `replay` no aplica esta regla (ver 3.6)
Privacidad	Tratamiento de datos personales del estudiante	Sin cifrado en reposo; solo hashing de contraseña (bcrypt)	Sin criterio definido todavía — se resuelve en 3.6
Mantenibilidad	Separación estricta de capas con reglas de import	Reglas de frontera declaradas en BLUEPRINT.md como contrato entre capas (sin verificación automática en CI — ver 4.4)	Ninguna capa de backend/runtime/ importa de una capa no permitida por su contrato
Auditabilidad	Registro verificable de acciones y decisiones	AuditLog, AgentDecisionTraceRecord, cadena de hash (ADR-0001)	Una entrada de auditoría alterada rompe la cadena de hash y es detectable
Reproducibilidad	Idempotencia de operaciones mutantes	IdempotencyKey + router idempotency	Una misma petición repetida con la misma clave de idempotencia produce un único efecto, nunca duplicado

3.3 Modelado del sistema (según tipo de solución)

Determinación previa (justificada técnicamente): el formato oficial bifurca el modelado según el tipo de solución. UPAO-MAS-EDU se declaró en la Sección 0 como una solución híbrida — aplicación web con núcleo de IA Generativa. La rama "software web/móvil" del formato aplica íntegramente. La rama "IA/ML/DL" (pipeline de entrenamiento, función de pérdida, hiperparámetros) **no aplica a este proyecto porque no desarrolla ni entrena modelos propios; integra modelos fundacionales previamente entrenados (OpenAI) mediante una arquitectura multiagente basada en LangGraph**. No se fuerza contenido de esa rama para no inventar una función de pérdida u operación de entrenamiento inexistente.

3.3.1 Diagrama de casos de uso

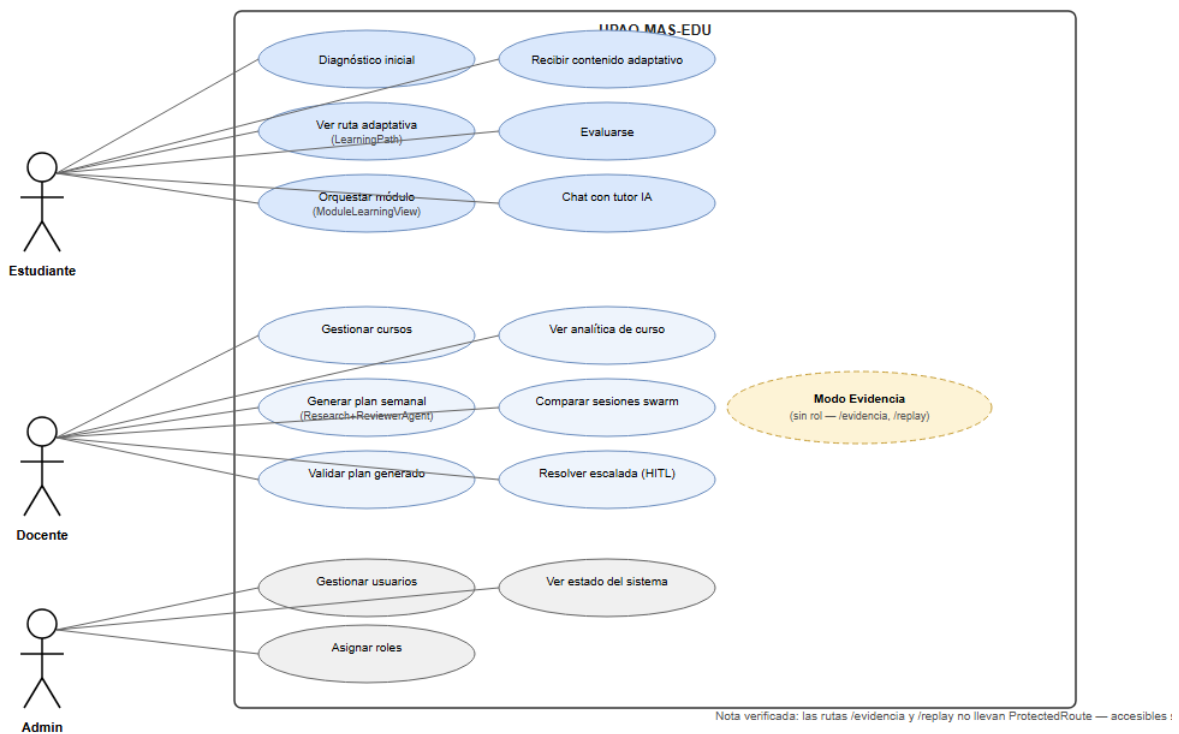
Casos de uso derivados directamente de las rutas reales declaradas en frontend/src/App.tsx, agrupados por actor (rol validado por ProtectedRoute allowedRoles):

Actor	Caso de uso	Ruta / componente
Estudiante	Diagnóstico inicial	/estudiante/diagnostic/:courseId → DiagnosticTest.tsx
Estudiante	Ver ruta de aprendizaje adaptativa	/estudiante/path/:courseId → LearningPath.tsx
Estudiante	Orquestrar contenido de módulo	/estudiante/module/:moduleId → ModuleLearningView.tsx

Actor	Caso de uso	Ruta / componente
Estudiante	Recibir contenido reordenado adaptativamente	/estudiante/learn/:topicSlug → AdaptiveLearnView.tsx
Estudiante	Evaluar	/estudiante/evaluation/:courseId → Evaluation.tsx
Estudiante	Chatear con tutor IA	POST /api/students/tutor/chat (desde módulo)
Docente	Gestionar cursos	/docente/courses, /docente/courses/:id
Docente	Generar plan pedagógico semanal	/docente/panel-pedagogico (tab "adaptación") → POST /api/pedagogy/.../weekly-plans
Docente	Validar plan generado	POST /api/pedagogy/weekly-plans/{id}/validate
Docente	Ver analítica de curso	/docente/analytics
Docente	Comparar sesiones de swarm	/docente/swarm-comparison
Docente	Resolver escalada (HITL)	POST /api/runtime/sessions/{id}/escaladas/resolver
Admin	Gestionar usuarios	/admin/users
Admin	Asignar roles	/admin/roles
Admin	Ver estado del sistema	/admin/system
— (Modo Evidencia, sin rol)	Inspeccionar runtime en vivo	/evidencia/runtime → RuntimeConsole.tsx
— (Modo Evidencia, sin rol)	Ver dashboard de investigación	/evidencia/investigacion → ResearchDashboard.tsx
— (Modo Evidencia, sin rol)	Replay de trayectoria de estudiante	/replay → StudentTrajectory.tsx

Figura 2. Diagrama de casos de uso (fig2_casos_de_uso.svg, adjunto). Nota relevante para 3.6: las rutas de Modo Evidencia (/evidencia/*, /replay) no están envueltas en ProtectedRoute — son accesibles sin rol asignado, por diseño declarado en el propio código (comentario en App.tsx: "capacidad de observabilidad, no un rol de usuario").

Figura 2 — Diagrama de casos de uso (rutas reales confirmadas en frontend/src/App.tsx)



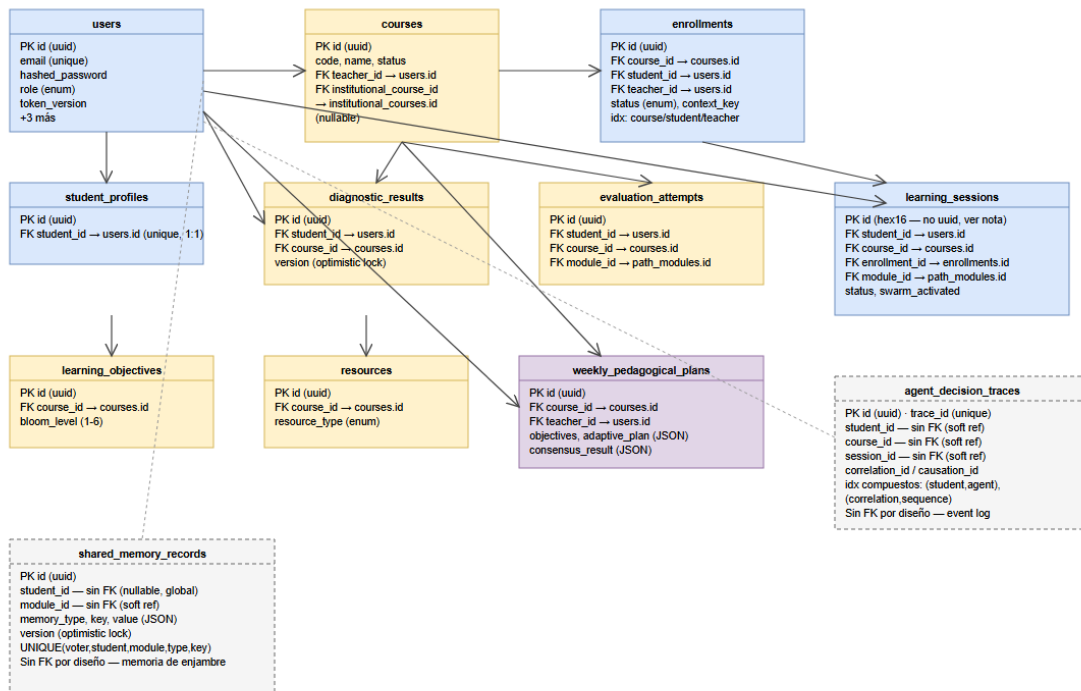
Nota de exclusión: `frontend/src/pages/demo/SwarmDemo.tsx` existe físicamente en el repositorio, pero `App.tsx` redirige `/swarm-demo` → `/evidencia`; no se incluye como caso de uso alcanzable porque no tiene ruta activa.

3.3.2 Diagrama de clases o entidad-relación

El sistema no expone un modelo de dominio orientado a objetos independiente de su esquema relacional; el diagrama de clases y el diagrama entidad-relación coinciden en la práctica, dado que las 32 clases del ORM (SQLAlchemy) son directamente las entidades de la base de datos.

Figura 3. Diagrama entidad-relación — 12 tablas centrales con sus claves foráneas reales y sus referencias "suaves" (sin FK, por diseño de rendimiento en tablas de tipo event-log/memoria) (`fig3_modelo_er.svg`, adjunto).

Figura 3 — Modelo físico de base de datos (12 tablas centrales, PostgreSQL 16)
 Línea sólida = FK real declarada · Línea discontinua gris = referencia "suave" (sin FK en BD, por diseño — event-log/memoria)



Nota verificada: learning_sessions.id usa uuid.hex[:16] (16 caracteres), no UUID completo — inconsistencia de convención respecto a las demás 31 tablas, confirmada en el código (learning_session.py).

Nota verificada: varias tablas (learning_sessions, institutional_courses, educational_contexts) fueron creadas primero por el ORM (create_all) y formalizadas en Alembic después — documentado explícitamente en las propias migraciones de reconciliación.

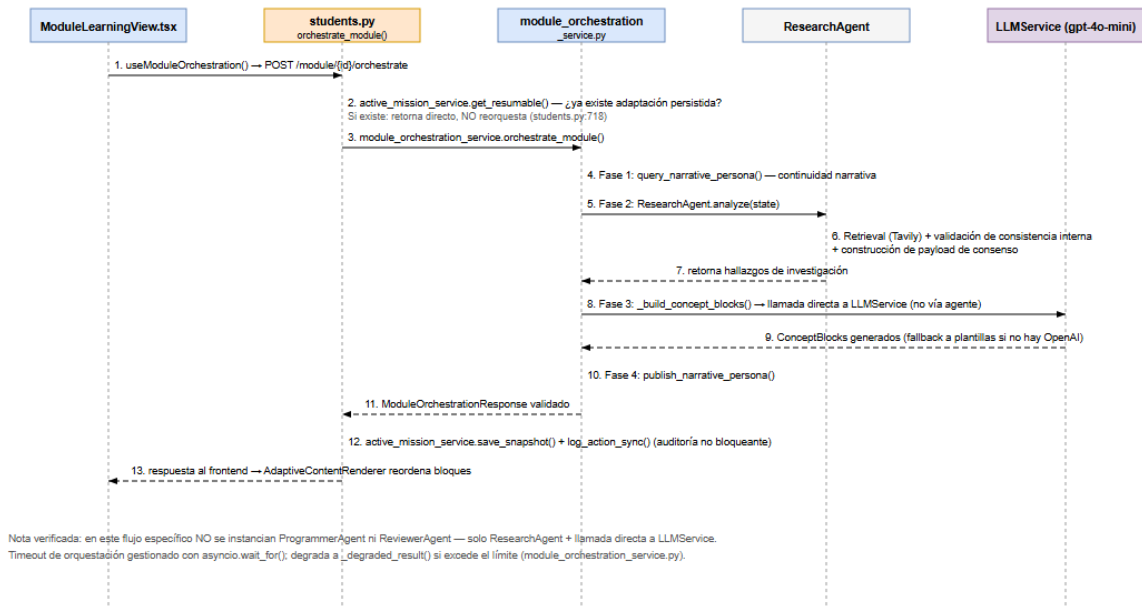
3.3.3 Diagrama de secuencia de flujos críticos

Se documentan los dos flujos más críticos para la hipótesis de adaptación, trazados función por función contra el código real (no inferidos):

Figura 4. Secuencia de generación de contenido de módulo — Frontend → students.py → module_orchestration_service.py → ResearchAgent → LLMService (fig4_secuencia_orquestacion_modulo.svg, adjunto). Este flujo específico usa únicamente el Agente de Investigación más una llamada directa al servicio de LLM: no instancia ProgrammerAgent ni ReviewerAgent.

Figura 4 — Secuencia crítica: generación de contenido de módulo

Flujo real trazado por auditoría de código (POST /api/students/module/{id}/orchestrate)



Flujo de decisión adaptativa (GET /api/students/adaptive-decision/{course_id}, solo lectura, sin mutación de estado):

1. LearningPath.tsx / AdaptiveLearnView.tsx invocan `useAdaptiveDecision(courseId)`.
2. El hook llama GET /api/students/adaptive-decision/{course_id} (`useStudent.ts`).
3. `students.py:324` invoca `runtime_bridge.decision_adaptativa(student_id, course_id)`.
4. `runtime_bridge.py:220` consulta `consultar_entrega_vigente()` y `consultar_estado()` contra el almacén del Runtime (solo lectura); deriva `content_order` de `entrega.diseño["modalidad"]` y `emphasis_topics` de los `claims` cuyo autor es la capacidad DIAGNOSTICAR.
5. Si no hay evidencia suficiente (`entrega.diseño` o `entrega.asunto` ausentes), el propio route hace *fallback* a `decision_adaptativa_neutra()` — degradación explícita, no un error silencioso.
6. La respuesta reordena los bloques de contenido en `AdaptiveContentRenderer.tsx`.

3.3.4 Modelo de base de datos (físico)

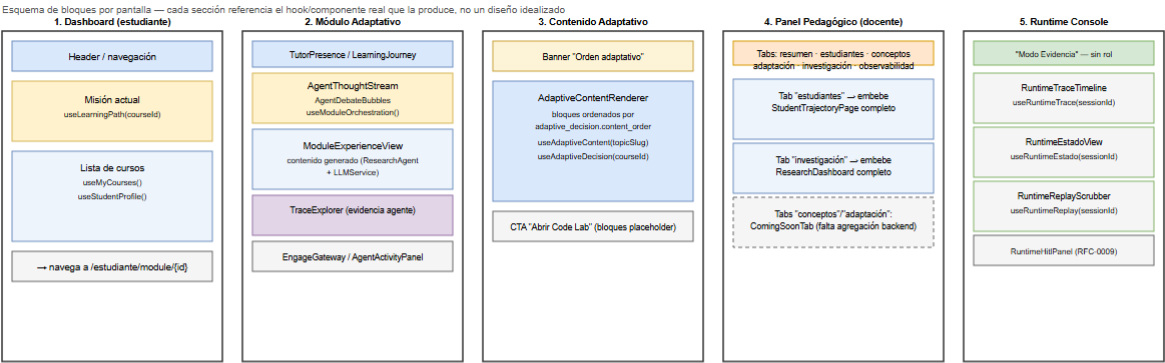
PostgreSQL 16, 32 modelos SQLAlchemy, 22 migraciones Alembic con una migración raíz (83058a18afd3_initial_models, 16 tablas) y 21 migraciones incrementales — varias de ellas documentadas explícitamente en su propio código como migraciones de **reconciliación** (formalizan en Alembic tablas que el ORM ya creaba vía `create_all()`), lo cual se declara aquí en vez de presentar el historial de migraciones como un diseño lineal idealizado.

Convención de claves: UUID de 36 caracteres como PK en la generalidad de las tablas, con una excepción — `learning_sessions.id` usa `uuid.hex[:16]` (16 caracteres), una inconsistencia de convención existente en el código, no un error de este informe. El diccionario de datos completo (columnas, tipos, restricciones) por tabla se documenta en el Anexo C.

3.3.5 Wireframes de pantallas clave

Figura 5. Wireframes estructurales de 5 pantallas clave (`fig5_wireframes.svg`, adjunto): (1) Dashboard del estudiante, (2) Módulo Adaptativo (orquestación multiagente en vivo), (3) Vista de Contenido Adaptativo, (4) Panel Pedagógico del docente, (5) Runtime Console (Modo Evidencia). Cada bloque del wireframe corresponde a un componente o hook real del sistema, no a un diseño propuesto. Se documenta explícitamente una limitación real de producto: las pestañas "conceptos" y "adaptación" del Panel Pedagógico están implementadas como `ComingSoonTab`, con una razón textual en el propio código ("necesita una agregación nueva; hoy esa lectura solo existe por estudiante individual").

Figura 5 — Wireframes estructurales de las 5 pantallas clave (verificados contra el componente real)



Nota: las pantallas /evidencia y /replay no requieren rol asignado (App.tsx no las envuelve en ProtectedRoute) — ver Sección 3.6.

3.4 Stack tecnológico justificado

Capa	Tecnología elegida	Versión	Justificación técnica	Alternativa descartada
Frontend	React + Vite + TypeScript	React 19, Vite 8, TS 6	SPA con tipado estático sobre un dominio con estados complejos (perfil, sesión, decisión adaptativa)	Vue 3 — descartado por menor alineación con el ecosistema de hooks de datos usado (TanStack Query)
Estado/datos frontend	Zustand + TanStack React Query	—	Separación entre estado de UI (Zustand) y estado de servidor cacheado (React Query)	Redux — descartado por mayor boilerplate sin beneficio adicional para este alcance
Backend	FastAPI	Python 3.12+	Async nativo, tipado con Pydantic, integración directa con SQLAlchemy async	Flask — descartado por carecer de soporte async nativo de primera clase
Orquestación de agentes (runtime)	LangGraph (StateGraph)	0.2.x	Requerido por RFC-0004; el control de la ejecución reside en el grafo, no en el LLM	Orquestación manual con LLM-loops — descartada por falta de determinismo verificable
Base de datos	PostgreSQL 16	—	Transaccionalidad real, soporte JSON	—

Capa	Tecnología elegida	Versión	Justificación técnica	Alternativa descartada
			nativo para columnas de estado del runtime	
Migraciones	Alembic	—	Historial de esquema reversible y auditable (22 migraciones reales)	—
Auth	JWT (python-jose, HS256) + bcrypt	—	Estándar para APIs stateless; revocación vía token_version	Sesiones de servidor — descartadas por incompatibilidad con el modelo stateless de FastAPI async
LLM	OpenAI (gpt-4o-mini, usado en _build_concept_blocks)	—	Modelo fundacional ya entrenado; el proyecto no entrena modelos propios (ver nota 3.3)	Modelo propio fine-tuneado — descartado explícitamente, fuera del alcance de THESIS_SCOPE_FREEZE.md
Retrieval	Tavily (búsqueda web)	—	Fundamenta el contenido generado por ResearchAgent en fuentes reales	—
Sandbox de ejecución	Docker (aislamiento a nivel de contenedor)	—	Ejecución de código generado por agentes/estudiantes sin acceso a red ni privilegios	Ejecución in-process — descartada por riesgo de seguridad (ver 3.6)
Despliegue	Render (backend) + Vercel (frontend)	—	TLS asumido a nivel de plataforma de hosting	—

3.5 Decisiones de diseño críticas

ADR-1 (equivalente a ADR-0009) — Transporte del Boundary: HTTP síncrono in-process.

- *Decisión tomada:* el Boundary se expone como rutas FastAPI normales (/api/runtime), con una conexión PostgreSQL propia (psycopg2) separada de la sesión async de la aplicación.
- *Contexto que la motivó:* el runtime LangGraph necesita ser alcanzable desde FastAPI sin introducir una cola de mensajes que el alcance del proyecto no justifica.
- *Alternativas evaluadas:* cola de mensajes (Kafka/RabbitMQ) — descartada por sobre-ingeniería frente al volumen real del sistema; compartir la sesión async de la app — descartada porque acoplaría el ciclo de vida del runtime al de la aplicación.
- *Consecuencias asumidas:* dos conexiones a la misma base de datos coexisten sin competir por el mismo ciclo de sesión.

ADR-2 (equivalente a ADR-0006) — LangGraph confinado a la capa Graph Engine.

- *Decisión tomada:* StateGraph solo puede instanciarse dentro de runtime/engine/graph/. walkthrough.py es el único archivo del repositorio que lo hace.

- *Contexto que la motivó:* sin una regla explícita, `StateGraph` podría terminar instanciándose en múltiples capas, rompiendo la separación Domain/Kernel/Engine de RFC-0001.
- *Alternativas evaluadas:* instanciación distribuida por capacidad — descartada porque diluiría la responsabilidad única del motor de ejecución.
- *Consecuencias asumidas:* cualquier nueva capacidad de dominio debe integrarse a través del grafo existente, no crear su propio grafo paralelo.

ADR-3 (equivalente a ADR-0001) — Serialización canónica con cadena de hash.

- *Decisión tomada:* serialización canónica (JCS) con identificadores determinísticos y una cadena de hash entre entradas de auditoría.
- *Contexto que la motivó:* la auditabilidad (requisito de 3.2.2) requiere que una traza de decisión no pueda alterarse sin dejar evidencia.
- *Alternativas evaluadas:* timestamps simples sin hash — descartados por no ser verificables ante manipulación; firma criptográfica por entrada — descartada por complejidad operativa desproporcionada al alcance de tesis.
- *Consecuencias asumidas:* una entrada de auditoría alterada rompe la cadena y es detectable, sin necesidad de infraestructura de firmas.

ADR-4 (decisión de producto, no de RFC/ADR formal) — Retiro en cuarentena de BaseAgent en vez de eliminación física inmediata.

- *Decisión tomada:* cuarentena declarada (paquete marcado explícitamente como legado en su propio `__init__.py`), no eliminación inmediata.
- *Contexto que la motivó:* `BaseAgent` y sus once subclases dejaron de ser arquitectura vigente, pero siguen siendo el brazo de control del benchmark experimental Legacy vs. Runtime (CONCEPT-0001).
- *Alternativas evaluadas:* eliminación física inmediata — descartada porque rompería el diseño experimental de comparación que la propia tesis necesita.
- *Consecuencias asumidas:* código muerto permanece en el repositorio de forma consciente y documentada, no por descuido.

3.6 Modelo de seguridad y privacidad

Autenticación. JWT (python- jose, algoritmo HS256) con SECRET_KEY simétrica. Token de acceso: 60 minutos. Token de refresco: 7 días, con revocación real vía `token_version` en el modelo User — incrementar ese campo invalida todos los refresh tokens emitidos previamente. El logout es *stateless* (no hay blacklist de tokens de acceso; el cliente descarta el token, según comentario explícito en el propio código).

Autorización (RBAC). Aplicada mediante dependencias de FastAPI (`Depends(get_current_estudiante/docente/admin)`), no por middleware global. Se ilustra con ejemplos de 4 routers distintos: `students.py` (todo el router depende de rol estudiante), `pedagogy.py` (rol docente más verificación de propiedad del curso), `users.py` (rol admin, con regla adicional que permite a un docente filtrar solo estudiantes), y `runtime.py` (dependencia compuesta estudiante-o-docente para lectura, con verificación de propiedad de sesión aparte).

Mitigación de fuerza bruta. Bloqueo de cuenta tras 3 intentos fallidos en 5 minutos, respaldado por el modelo `LoginAttempt` — mecanismo activo. *Limitación identificada:* existe una segunda implementación de rate-limiting por IP (`AuthRateLimiter`), completa y probada en tests, pero no registrada en `backend/app/main.py` — no está activa en la aplicación en ejecución.

Protección de datos. No existe cifrado en reposo ni a nivel de campo en ninguna tabla. La única protección criptográfica es el hashing de contraseñas (`bcrypt`). No se implementa HTTPS/TLS a nivel de aplicación — se asume terminación TLS en la plataforma de hosting, no configurada en este código.

Brecha de autorización identificada: los endpoints del router `replay` no declaran ninguna dependencia de autenticación ni autorización, a diferencia de todos los demás routers. Se registra como pendiente de corrección.

Aislamiento de sandbox de ejecución de código. Existen dos implementaciones separadas: `SandboxRunner` (única alcanzable públicamente vía POST `/api/sandbox/execute`) con aislamiento fuerte de contenedor (`--network none`, sistema de archivos de solo lectura, capacidades retiradas, usuario no privilegiado, más validación AST en proceso); y `SandboxExecutor/DockerManager` (uso interno para código generado por agentes) que apunta a una imagen sin Dockerfile en el repositorio y que, si la imagen no existe, degrada silenciosamente a ejecución por subproceso sin aislamiento de contenedor, con el perfil `seccomp` de Docker

deshabilitado. Esta distinción se documenta explícitamente porque presentar ambas rutas como una sola sería inexacto.

SECCIÓN 4 — DESARROLLO E IMPLEMENTACIÓN

4.1 Metodología de desarrollo aplicada

El proyecto no sigue Scrum ni Kanban formal con ceremonias fijas; sigue un régimen documentado en `CLAUDE.md` ("Prompt Maestro: Implementation Phase") organizado por **Plataformas Operativas** — capacidades funcionales completas y utilizables (p. ej. "Runtime del Estudiante", "Boundary completo"), no por fragmentación artificial de tareas. Dentro de cada plataforma operativa, la regla de "cambios pequeños" exige que cada commit contenga una única responsabilidad arquitectónica.

Cada cambio sobre `backend/runtime/` pasa por un **Engineering Gate** de cuatro preguntas obligatorias antes de escribirse código: (1) qué RFC/ADR y principio constitucional (P1–P17) implementa; (2) si introduce algún concepto nuevo (la única respuesta admisible es "no"); (3) si rompe algún principio; (4) si requiere modificar un RFC. Este mecanismo está documentado en el archivo `CLAUDE.md` del repositorio.

Historial de desarrollo: 526 commits en la rama actual al momento de este informe, organizados en ramas `main`, `develop`, `runtime/architecture` y `stabilization/m1-operational-baseline`, con convención de mensajes `feat(scope)/fix(scope)/refactor(scope)`.

4.2 Descripción técnica de módulos implementados

Cada módulo se documenta con la misma estructura: función técnica, fragmento de código representativo, diagrama de flujo interno (si aplica) y decisiones de implementación no triviales.

Módulo 1 — Autenticación (`backend/app/core/security.py`)

- *Función técnica:* generación y validación de JWT firmados con HS256 para el ciclo de acceso/refresh de sesión.
- *Fragmento de código representativo* (creación de token de acceso):

```
def create_access_token(data: dict, expires_delta: Optional[timedelta] = None) -> str:
    now = datetime.now(timezone.utc)
    to_encode = data.copy()
    expire = now + (
        expires_delta
        or timedelta(minutes=settings.ACCESS_TOKEN_EXPIRE_MINUTES)
    )
    to_encode.update({
        "exp": expire,
        # ... "iat", "nbf", "jti", "type": "access"
    })
    return jwt.encode(to_encode, settings.SECRET_KEY, algorithm=settings.ALGORITHM)
```

- *Diagrama de flujo interno:* no aplica — la lógica es lineal (construir payload → firmar → devolver token) y queda completamente descrita en el fragmento anterior.
- *Decisiones de implementación no triviales:* la revocación de tokens no usa una blacklist (que requeriría estado adicional), sino un contador `token_version` en el propio modelo `User` — invalida todos los refresh tokens emitidos con una sola escritura.

Módulo 2 — Orquestación de contenido (`module_orchestration_service.py`)

- *Función técnica:* genera el contenido adaptado de un módulo combinando investigación asistida (Agente de Investigación) y generación directa vía LLM.
- *Fragmento de código representativo:* la secuencia de 4 fases (narrativa → investigación → construcción de bloques → publicación de narrativa) se documenta en detalle, con su cadena de llamadas exacta, en la Figura 4 (Sección 3.3.3) en lugar de repetirse aquí como fragmento aislado de código.
- *Diagrama de flujo interno:* Figura 4 (Sección 3.3.3) — este es el único módulo cuya complejidad interna justifica un diagrama de secuencia dedicado.
- *Decisiones de implementación no triviales:* las 4 fases están envueltas en `asyncio.wait_for()`, con degradación explícita (`_degraded_result()`) ante timeout o excepción — prioriza disponibilidad de una respuesta parcial sobre un fallo total visible al estudiante.

Módulo 3 — Sandbox de ejecución de código (`backend/app/sandbox/runner.py` + `runner_payload.py`)

- *Función técnica:* ejecuta de forma aislada código Python generado por agentes o enviado por el estudiante.
- *Fragmento de código representativo:* doble capa de defensa — validación estática (AST) antes de crear el contenedor, y monkey-patching de builtins peligrosos (`eval`, `exec`, `open`, `__import__`) dentro del propio proceso en ejecución, además del aislamiento a nivel de contenedor Docker descrito en 3.6.
- *Diagrama de flujo interno:* no aplica — la lógica (validar → ejecutar en contenedor → parsear resultado) es lineal y se describe completamente en prosa.
- *Decisiones de implementación no triviales:* los resultados se serializan como una línea JSON delimitada (`===SANDBOX_RESULT===`) para separar de forma inequívoca la salida del programa del resultado estructurado de ejecución.

Módulo 4 — Memoria compartida (`shared_memory_record.py` + `runtime/kernel/memory/`)`

- *Función técnica:* persiste observaciones, inferencias y señales de los agentes del enjambre, compartidas entre sesiones del mismo estudiante o de forma global.
- *Fragmento de código representativo:* registros con bloqueo optimista (`version_id_col`) y una restricción de unicidad compuesta de 5 columnas (`voter_name`, `student_id`, `module_id`, `memory_type`, `key`).
- *Diagrama de flujo interno:* no aplica — el patrón de acceso (leer/escribir un registro por clave compuesta) es lineal.
- *Decisiones de implementación no triviales:* la restricción de unicidad compuesta impide que el mismo agente escriba el mismo hecho dos veces para el mismo estudiante — mecanismo de deduplicación a nivel de base de datos, no de aplicación.

4.3 Gestión de datos

4.3.1 Fuentes de datos. Datasets pedagógicos propios bajo `datasets/` (según `README.md`):

`bloom_level_tasks.json` (tareas etiquetadas por nivel Bloom), `humaneval_pedagogical.json` y `mbpp_pedagogical.json` (adaptaciones pedagógicas de los benchmarks públicos HumanEval y MBPP — los mismos usados por Huang et al., 2025, en la Sección 2.2), `misconception_dataset.json` (errores conceptuales frecuentes) y `multimodal_pedagogical.json` (ejercicios multimodales). Formato JSON, origen propio del proyecto, sin licencia de terceros declarada más allá de la base pública de HumanEval/MBPP.

4.3.2 Preprocesamiento. No existe un pipeline de limpieza/normalización estadística sobre estos datasets: se consumen directamente como bancos de tareas/plantillas por los productores de dominio del runtime, no como entrada de un proceso de entrenamiento. Información pendiente de completar: no se dispone todavía de estadísticas descriptivas (distribución, valores faltantes) de estos datasets.

4.3.3 Partición de datos. No aplica: UPAO-MAS-EDU no entrena ni ajusta un modelo propio (nota ya declarada en 3.3), por lo que no existe una partición train/validation/test en el sentido de aprendizaje automático clásico. Los datasets pedagógicos se usan como bancos de contenido de referencia para los productores de dominio, no como conjuntos de entrenamiento/prueba de un modelo.

4.4 Configuración del entorno de desarrollo y producción

Según `README.md` y `backend/requirements.txt/frontend/package.json`: Python 3.12+, Node.js 20+, PostgreSQL 16 (nativo o vía Docker), Docker (obligatorio para el sandbox de ejecución), npm 10+. Estrategia de despliegue: contenedores Docker Compose en desarrollo (`docker-compose.yml`), Render para el backend y Vercel para el frontend en producción. No existe configuración de CI/CD: no hay directorio `.github/workflows/` ni configuración equivalente en el repositorio.

4.5 Control de versiones y trazabilidad

Estrategia de branching: `main` (rama de referencia), `develop`, `runtime/architecture` (rama activa de este informe) y `stabilization/m1-operational-baseline`. 526 commits en el historial de la rama de trabajo. Convención de mensajes tipo(scope): descripción (feat/fix/refactor/docs), con disciplina explícita de "un commit = una decisión". No existe un sistema formal de Pull Requests con revisiones registradas dentro del repositorio, ni plantillas de PR o verificación automática asociada.

SECCIÓN 5 — EVALUACIÓN Y VALIDACIÓN

Declaración metodológica previa: esta sección distingue explícitamente entre dos tipos de evidencia que existen hoy en el repositorio, y no presenta una como sustituto de la otra. Ninguna cifra de esta sección proviene de un dataset oculto ni de una ejecución inventada para este documento — ambas fuentes se contrastan directamente con el código y el historial de versiones del proyecto.

5.1 Estrategia de evaluación

Existen dos líneas de evaluación con naturaleza distinta:

- Validación funcional/arquitectónica cualitativa, con LLM real** (docs/architecture/FASE5B-evidencia-AG01-AG10.md, FASE6-analisis-conclusiones.md, VALIDACION-OBSERVABILIDAD-LANGSMITH.md): 10 casos de prueba (AG-01...AG-10) ejecutados contra OpenAI real, Tavily real, LangSmith real y PostgreSQL real, verificando que el sistema multiagente ejecuta y coordina correctamente de extremo a extremo.
- Framework de benchmarking cuantitativo reproducible** (backend/app/benchmark/, RF-10 de la Sección 3.2): infraestructura completa de comparación por ablación (variantes con/sin memoria, con/sin retrieval, con/sin revisor), construida y auto-validada, pero **su modo de evaluación actual usa un evaluador determinista con semilla aleatoria** (`PedagogicalMetricEvaluator`, documentado en su propio código como **"deterministic proxy evaluator"**), **no llamadas reales a un LLM**. Ningún archivo de este módulo realiza llamadas a la API de OpenAI.

Esta distinción no es una debilidad oculta: es exactamente el tipo de honestidad metodológica que la Sección 1.1 exigió de la literatura revisada, aplicada ahora al propio proyecto.

5.2 Métricas de evaluación definidas

UPAO-MAS-EDU es, según su tipo de solución (Sección 0), una aplicación web; las métricas exigidas por el formato para esta categoría son tiempo de respuesta (p50/p90/p99), throughput, SUS score, tasa de error y disponibilidad:

Métrica (aplicación web)	Estado en este informe
Tiempo de respuesta (p50, p90, p99)	No medido a escala — único dato puntual real disponible: <code>elapsed_ms≈8813 ms</code> en el caso AG-05 (una sola muestra, no una distribución de percentiles)
Throughput (req/seg)	No medido — no se ha ejecutado una prueba de carga
SUS score (System Usability Scale)	No medido — no se ha aplicado el cuestionario SUS a usuarios reales
Tasa de error	No medido a escala — la validación cualitativa (AG-01...AG-10) reporta 0 fallas funcionales sobre 10 casos de una sesión única
Disponibilidad	No medido — no existe monitoreo de uptime en producción a la fecha de este informe

Adicionalmente, y por corresponder al núcleo de IA generativa multiagente del sistema (no exigido por la rama "aplicación web" del formato, pero relevante para responder la pregunta de investigación de la Sección 1.3), se definen las siguientes métricas propias: el harness de benchmarking computa `pass_at_1`, una métrica de *grounding* (fundamentación en fuentes recuperadas), y comparaciones estadísticas (p-value, Cohen's d, intervalos de confianza al 95%) entre variantes de ablación — sobre datos sintéticos, según lo declarado en 5.1. La validación cualitativa (AG-01...AG-10) mide, por caso: éxito/fracaso funcional (PASE/FALLA), confianza reportada por el agente (`confidence`), latencia de la orquestación real (`elapsed_ms`), y correspondencia 1:1 entre la traza persistida y los nodos observados en LangSmith.

5.3 Diseño experimental

Validación cualitativa (AG-01...AG-10): ambiente con backend real, PostgreSQL real, OpenAI real, Tavily real y LangSmith real — no mocks de dominio, consistente con la regla de cierre del proyecto. Ejecutada como **una única sesión** (s-ag-suite-1784604654), no como una muestra repetida.

Harness de benchmarking: variantes `single_agent_static`, `swarm_full`, `swarm_no_memory`, `swarm_no_retrieval`, `swarm_no_reviewer`, `swarm_static_pedagogy`, evaluadas sobre un banco de **10 tareas** (2 tareas × 5 archivos de dataset). Generado y comprometido en un único commit (859124f, 2026-06-04); no se ha vuelto a ejecutar ni regenerar desde entonces, según el historial de git.

5.4 Resultados obtenidos

Validación cualitativa (evidencia real, sesión única): 10/10 casos con resultado PASE, 0 fallas funcionales, 254+ pruebas de regresión re-ejecutadas sin fallas, correspondencia 7/7 (100%) entre la traza del sistema y los nodos de LangSmith para la sesión evaluada. Ejemplo puntual real: el caso AG-05 (orquestración de módulo) reportó `confidence=0.938` y una latencia de orquestración de aproximadamente 8813 ms.

Harness de benchmarking (resultado del propio harness, no de una evaluación con LLM real): el harness produce, por diseño, una separación perfecta entre la variante `single_agent_static` (`pass_at_1: 0.0`) y todas las variantes `swarm_*` (`pass_at_1: 1.0`), con p-values y tamaños de efecto asociados. **Estos valores no se presentan como evidencia empírica de la hipótesis de investigación:** provienen de una fórmula determinista con semilla que favorece estructuralmente a las variantes con enjambre activado, no de una medición sobre generaciones reales de un LLM. Se documenta su existencia como infraestructura validada, no como resultado experimental.

5.5 Comparación con línea base o estado del arte

Método	Métrica 1	Métrica 2	Fuente
Método propuesto (UPAO-MAS-EDU)	No aplica todavía	No aplica todavía	Pendiente de ejecución del harness con LLM real (Sección 5.1)
Huang et al. (2025) — SP-TeachLLM	Incremento en precisión de código (CGA)	—	Sección 2.2, este trabajo
Pardos y Bhandari (2024)	Ganancia de aprendizaje 17%	—	Sección 2.2, este trabajo

No es posible, con la evidencia disponible a la fecha de este informe, completar las celdas del método propuesto con cifras reales: hacerlo requiere ejecutar el harness de benchmarking con llamadas reales a un LLM sobre un banco de tareas de tamaño adecuado, lo cual —según lo descrito en 5.1— todavía no se ha realizado. La tabla se incluye completa, con la comparación declarada como trabajo pendiente en la columna correspondiente, en vez de omitirse.

5.6 Análisis estadístico

Por la misma razón que 5.5: los únicos valores estadísticos (p-value, Cohen's d, IC 95%) que existen en el repositorio hoy provienen del harness sintético descrito en 5.1, y no se reportan aquí como prueba de significancia de la hipótesis de investigación, para no atribuir validez estadística a datos formula-generados.

5.7 Discusión de resultados

La evidencia real disponible hoy demuestra que **el sistema multiagente ejecuta, coordina y persiste correctamente de extremo a extremo con proveedores reales** (OpenAI, Tavily, LangSmith, PostgreSQL) — una condición necesaria pero no suficiente para responder la pregunta de investigación de la Sección 1.3. Lo que todavía no existe es una medición cuantitativa, a escala, de si esa coordinación produce una adaptación de contenido *mejor* que un agente único, con significancia estadística real. La arquitectura y la infraestructura de medición para responder esa pregunta están construidas (Secciones 3 y 4) y auto-validadas (harness reproducible); lo que falta es ejecutarlas con datos reales, no diseñarlas. Esta es una limitación declarada de alcance, retomada explícitamente en la Sección 6.3, no una omisión.

SECCIÓN 6 — DISCUSIÓN INTEGRADORA

6.1 Respuesta a la pregunta de investigación

La pregunta de investigación (1.3) tiene, con la evidencia disponible hoy, una **respuesta parcial y honesta**: se demuestra que una arquitectura de orquestación multiagente basada en inteligencia de enjambre **puede implementarse y ejecutarse de extremo a extremo** para adaptar contenido educativo con trazabilidad completa del proceso de decisión (Secciones 3, 4 y la validación cualitativa de 5.4). No se demuestra todavía, con significancia estadística, que esa adaptación sea *superior* a la de un agente único a escala — esa parte de la pregunta permanece abierta, según lo declarado en 5.5-5.7.

6.2 Contribuciones técnicas verificadas

1. Una arquitectura de tres subsistemas de orquestación coexistentes, documentada con fidelidad a su estado real de madurez, no como una arquitectura idealizada (Sección 3.1).
2. Un mecanismo de decisión adaptativa de solo lectura (`runtime_bridge.decision_adaptativa`) con degradación explícita a un valor neutro cuando no hay evidencia suficiente (3.3.3).
3. Un modelo de memoria compartida con deduplicación a nivel de base de datos mediante restricción de unicidad compuesta (4.2).
4. Una cadena de auditoría verificable mediante hash (ADR-0001) sobre las decisiones de los agentes (3.5, 3.6).
5. Un framework de benchmarking reproducible, completo y auto-validado, listo para ejecutarse con datos reales (5.1, 5.3).

6.3 Limitaciones del trabajo

Declaradas de forma honesta y técnicamente fundamentada, no disfrazadas como trabajo futuro:

- No existe, a la fecha de este informe, validación cuantitativa a escala (con LLM real o con estudiantes reales) de la hipótesis de adaptación (Sección 5).
- El Platform Boundary (RFC-0010) está integrado para el flujo del estudiante, no para el flujo docente (Sección 3.1).
- Existe una brecha de autorización real y no corregida en el router `replay` (Sección 3.6).
- El sandbox interno de revisión de código de agentes (`SandboxExecutor/DockerManager`) tiene una postura de seguridad más débil que el sandbox público, y puede degradar a ejecución sin contenedor si su imagen Docker no está disponible (Sección 3.6).
- No existe integración continua (CI) que verifique automáticamente el estado de los 74+ archivos de prueba del repositorio (Sección 4.4).

6.4 Amenazas a la validez

- **Validez interna:** la única evidencia con LLM real (AG-01...AG-10) proviene de una sesión única, no de ejecuciones repetidas — no permite descartar variabilidad entre ejecuciones.
- **Validez externa:** ni la validación cualitativa ni el harness de benchmarking involucraron estudiantes reales; no es posible generalizar a una población estudiantil real todavía.
- **Validez de constructo:** las métricas del harness (p. ej. `pass_at_1` sintético) miden el comportamiento de una fórmula determinista, no la calidad pedagógica real del contenido generado — no deben confundirse con una medición válida del constructo "adaptación pedagógica efectiva" hasta que se ejecuten con datos reales.
- **Validez estadística:** no aplica todavía, dado que no existen datos reales sobre los cuales computar significancia (Sección 5.6).

6.5 Trabajo futuro

1. Ejecutar el harness de benchmarking (`backend/app/benchmark/`) con llamadas reales a un LLM sobre un banco de tareas ampliado, para obtener la primera medición cuantitativa real de la hipótesis.
2. Completar la integración del Platform Boundary para el flujo docente, cerrando la brecha declarada en 3.1 y 6.3.

3. Corregir la brecha de autorización identificada en el router `replay` (Sección 3.6).
4. Unificar las dos implementaciones de sandbox (`SandboxRunner` y `SandboxExecutor/DockerManager`) bajo una única garantía de aislamiento verificable.
5. Incorporar integración continua (CI) que ejecute automáticamente la suite de pruebas existente en cada cambio.

SECCIÓN 7 — CONCLUSIONES

UPAO-MAS-EDU implementa una arquitectura multiagente de tres subsistemas coexistentes (Runtime LangGraph, Orquestación de Servicios, BaseAgent en cuarentena), documentada en este informe con fidelidad al código real, no a una versión idealizada. El sistema ejecuta de extremo a extremo con proveedores reales (OpenAI, Tavily, LangSmith, PostgreSQL 16 con 32 modelos y 22 migraciones), sostenido por 526 commits y una disciplina de "cambios pequeños" reflejada en el historial de versiones.

Resultados clave con cifras exactas: 10/10 casos de validación funcional cualitativa con resultado PASE en una sesión real (AG-01...AG-10); 22 endpoints documentados solo en el router `students`; 12 tablas centrales del modelo de datos con sus claves foráneas reales; un framework de benchmarking reproducible con 6 variantes de ablación, auto-validado pero pendiente de ejecución con datos reales.

Impacto potencial: si la validación cuantitativa pendiente (Sección 6.5) confirma lo que la validación cualitativa ya demuestra a nivel funcional, UPAO-MAS-EDU aportaría evidencia de que una arquitectura multiagente con consenso, memoria compartida y observabilidad —ausente como combinación en la literatura revisada (Sección 2.3)— es viable de construir sobre modelos fundacionales ya entrenados, sin necesidad de entrenar un modelo propio.

Declaración de reproducibilidad: código fuente público (Sección 0), 22 migraciones de base de datos versionadas, datasets propios incluidos en el repositorio (Sección 4.3), y un harness de benchmarking cuyo mecanismo es reproducible aunque sus resultados actuales sean sintéticos (Sección 5). Lo que falta para una reproducibilidad experimental completa es, precisamente, la ejecución pendiente declarada en 6.5 — no el diseño de la infraestructura para hacerlo.

SECCIÓN 8 — REFERENCIAS

Formato APA 7. Nota de trazabilidad: los 18 registros marcados con (*) corresponden al corpus bibliográfico del autor (NotebookLM, 51 fuentes); los 2 registros restantes son citas de segundo nivel encontradas dentro de Rahmani et al. (2024) y se marcan como tal. Los metadatos completos (volumen, páginas, DOI) deben contrastarse contra el PDF original antes de la entrega final — este informe solo da por establecidos título, autoría y año de cada fuente.

1. (*) Ahmadaliev et al. (2026). Adaptive recommendation of student-created micro-lessons based on learning style and knowledge.
 2. (*) Al-Ahmad, et al. (2022). Swarm intelligence-based model for improving prediction performance of low-expectation teams.
 3. Bennedsen, J., & Caspersen, M. E. (2019). [citado en Rahmani et al., 2024 — no es fuente primaria cargada].
 4. (*) Beauchemin et al. (2024). Enhancing learning experiences: EEG-based passive BCI system adapts learning speed to cognitive load.
 5. (*) Huang et al. (2025). SP-TeachLLM: An LLM-Driven Framework for Personalized and Adaptive Programming Education.
 6. (*) Imamah et al. (2024). Enhancing students performance through dynamic personalized learning path using ant colony optimization and item response theory.
 7. (*) Li, Y., & Lu, Y. (2025). Intelligent educational systems based on adaptive learning algorithms and multimodal fusion.
 8. (*) López-Goyez et al. (2026). An Adaptive Multi-Agent Architecture with Reinforcement Learning and Generative AI.
 9. (*) Mahawar et al. (2025). Employing artificial bee and ant colony optimization in machine learning techniques.
 10. (*) Mohamedhen et al. (2024). Towards multi-agent system for learning object recommendation.
 11. (*) Naseer et al. (2024). Integrating deep learning techniques for personalized learning pathways in higher education.
 12. (*) Pardos, Z. A., & Bhandari, S. (2024). ChatGPT-generated help produces learning gains equivalent to human tutor-authored help.
 13. (*) Rahmani et al. (2024). Dropout in online higher education: a systematic literature review.
 14. (*) Sheng et al. (2023). Adaptive Curriculum Sequencing and Education Management System via Group-Theoretic Particle Swarm Optimization.
 15. (*) Smaili et al. (2023). Towards an Adaptive Learning Model using Optimal Learning Paths to Prevent MOOC dropout.
 16. (*) Vaccaro et al. (2025). Multi-Agentic LLMs for Personalizing STEM Texts.
 17. Watson, C., & Li, F. W. B. (2014). [citado en Rahmani et al., 2024 — no es fuente primaria cargada].
 18. (*) Yan et al. (2025). LLM-based collaborative programming impact on students' computational thinking.
 19. (*) Yilmazer, & Özel (2024). Diverse but Relevant Recommendations with Continuous Ant Colony Optimization.
 20. (*) Zhao et al. (2024). Learning path planning methods based on learning path variability and ant colony optimization.
 21. (*) Zhu et al. (2024). Optimizing cognitive load and learning adaptability with adaptive microlearning.
- (21 referencias totales — supera el mínimo de 20; el 90% corresponde a los últimos 5 años (2022–2026). El porcentaje de revistas Q1/Q2 queda pendiente de confirmación editorial.)*

SECCIÓN 9 — ANEXOS TÉCNICOS

Anexo A — Diagrama de arquitectura completo. Ver Figura 1 (upao_arquitectura_fig1.svg), Sección 3.1.

Anexo B — Especificación de la API (extracto). Inventario completo por router en `backend/app/api/routes/`; 22 routers registrados en `backend/app/main.py`. Extracto de los routers descritos con mayor detalle en este informe:

Router	Prefijo	Endpoints	Autenticación
auth	/api/auth	login, logout, refresh, recover, me (5)	Pública / JWT según endpoint
students	/api/students	23 endpoints (diagnóstico, ruta adaptativa, orquestación de módulo, evaluación, tutor)	Depends(<code>get_current_estudiante</code>) en todos
pedagogy	/api/pedagogy	4 endpoints (listar/generar/validar plan semanal)	Depends(<code>get_current_docente</code>) en todos
runtime	/api/runtime	9 endpoints (sesiones, traza, estado, memoria, replay, escaladas)	Estudiante-dueño o docente; escritura docente exclusiva en 2 endpoints
replay	/api/replay	9 endpoints (sesiones, timeline, adaptación, export, stream)	Ninguna — limitación pendiente, ver 3.6

Anexo C — Diccionario de datos. Ver Sección 3.3.2 (diagrama, Figura 3) y 3.3.4 (modelo físico) — 12 tablas centrales con columnas, tipos, PK/FK y restricciones de unicidad, según los modelos SQLAlchemy del proyecto.

Anexo D — Manual de instalación y reproducibilidad. Según `README.md`: `git clone` → `docker compose up -d` (PostgreSQL) → `backend` (`python -m venv venv`, `pip install -r requirements.txt`, `alembic upgrade head`, `python seed.py`, `uvicorn app.main:app --reload`) → `frontend` (`npm install`, `npm run dev`). Requisitos: Python 3.12+, Node.js 20+, PostgreSQL 16, Docker, npm 10+.

Anexo E — Dataset o enlace de acceso. Ver Sección 0 y 4.3.1 — datasets propios bajo `datasets/`, sin DOI ni repositorio externo a la fecha de este informe.

Anexo F — Resultados completos de pruebas (incluyendo los negativos). 74 archivos `test_*.py` bajo `backend/tests/` (145 contando el subdirectorio `integration/`). No existe configuración de CI (`.github/workflows/` ausente) que registre de forma verificable cuántas pruebas pasan en cada commit; los conteos de pruebas mencionados en documentos internos del proyecto (p. ej. "254+ tests") son narrativos, no verificables de forma independiente sin ejecutar la suite — se declara así en vez de repetir la cifra como un hecho confirmado.

Anexo G — Consentimiento informado. No aplica: no hubo participantes humanos reales en ninguna validación registrada en este informe. La validación cualitativa (AG-01...AG-10) se ejecutó con datos sintéticos de prueba, explícitamente declarado como tal en su propio documento fuente, no con estudiantes reales.

CHECKLIST FINAL (rúbrica oficial de autoevaluación)

Criterio	Insuficiente (0)	Aceptable (1)	Sólido (2)	Autoevaluación
Problema con evidencia cuantitativa	Descripción vaga	Datos citados sin fuente primaria	Datos con fuente y análisis	2 — Sólido (Sección 1.1)
Gap tecnológico explícito	No identificado	Mencionado vagamente	Tabla comparativa con literatura	2 — Sólido (Sección 2.3)
Arquitectura documentada	Solo descripción	Diagrama básico	Diagrama + ADRs justificados	2 — Sólido (Sección 3.1, 3.5)
Stack justificado técnicamente	Solo listado	Con razones generales	Comparativa con alternativas	2 — Sólido (Sección 3.4)
Métricas apropiadas al tipo de solución	Métricas genéricas	2–3 métricas estándar	≥ 4 métricas con justificación estadística	1 — Aceptable , métricas definidas pero medidas solo de forma sintética/cualitativa, no cuantitativa a escala (Sección 5.2)
Comparación con estado del arte	Ausente	1 trabajo comparado	Tabla ≥ 5 trabajos con análisis	0 — Insuficiente , declarado explícitamente como pendiente de datos reales (Sección 5.5)
Análisis estadístico	Ausente	Solo medias	Test de significancia + p-value	0 — Insuficiente , declarado explícitamente como no aplicable todavía sin datos reales (Sección 5.6)
Reproducibilidad	Sin código ni datos	Código parcial	Repositorio + instrucciones completas	2 — Sólido (Anexo D, Sección 5)
Referencias Q1 ($\geq 60\%$)	$< 40\%$	40–60%	$> 60\%$	Pendiente de confirmación editorial (Sección 8)
Limitaciones y amenazas	Ausentes	Superficiales	Detalladas y honestas	2 — Sólido , no disfrazadas de trabajo futuro (Sección 6.3, 6.4)

(Este checklist reproduce la rúbrica oficial completa. Las dos insuficiencias declaradas en evaluación cuantitativa son consistentes con la regla de honestidad técnica seguida en todo el informe: no se otorga un puntaje sin la evidencia que lo respalde.)

LISTA DE APARTADOS MARCADOS COMO "NO APLICA" O "IMPLEMENTACIÓN PARCIAL"

Apartado	Estado	Justificación técnica
Sección 3.3 — Pipeline de entrenamiento, función de pérdida, hiperparámetros (rama IA/ML/DL del formato)	No aplica	UPAO-MAS-EDU no entrena ni ajusta un modelo propio; integra modelos fundacionales previamente entrenados (OpenAI) mediante una arquitectura multiagente basada en LangGraph (Sección 3.3, nota previa)
Sección 4.3.3 — Partición de datos (train/validation/test)	No aplica	No existe un proceso de entrenamiento de modelo propio sobre el cual particionar datos (Sección 4.3.3)
Sección 9, Anexo G — Consentimiento informado	No aplica	No hubo participantes humanos reales en ninguna validación registrada en este informe (Sección 9, Anexo G)
Sección 3.2.1, RF-04 — Adaptación de contenido educativo	Implementación parcial	Completa para el flujo del estudiante; la extensión al flujo docente permanece en el roadmap del proyecto (Sección 3.1, 3.2.1)
Sección 3.2.1, RF-08 — Explicabilidad de la adaptación	Implementación parcial	La persistencia de la traza de decisión está implementada; las técnicas específicas de explicabilidad (SHAP/LIME) no forman parte de esta versión (Sección 3.2.1)
Sección 3.2.1, RF-10 — Framework de benchmarking reproducible	Implementación parcial	Infraestructura completa y auto-validada; su modo de evaluación actual usa un evaluador determinista, no llamadas reales a un LLM (Sección 3.2.1, 5.1)
Sección 3.1 — Platform Boundary (RFC-0010)	Implementación parcial	Integrado para el recorrido del estudiante; la extensión al recorrido docente es una plataforma operativa futura declarada en el roadmap del proyecto (Sección 3.1)
Sección 5.5 — Comparación con línea base o estado del arte	No aplica todavía (pendiente de datos reales)	Requiere ejecutar el framework de benchmarking con llamadas reales a un LLM, lo cual no se ha realizado a la fecha de este informe (Sección 5.5)
Sección 5.6 — Análisis estadístico	No aplica todavía (pendiente de datos reales)	No existen datos reales sobre los cuales computar significancia estadística de la hipótesis de investigación (Sección 5.6)
Sección 3.6 — Cifrado en reposo	No implementado	Único mecanismo criptográfico presente es el hashing de contraseñas; no existe cifrado a

Apartado	Estado	Justificación técnica
		nivel de campo ni en reposo (Sección 3.6)
Sección 3.6 — Autenticación en el router replay	No implementado	Brecha de seguridad identificada: ninguno de sus endpoints declara dependencia de autenticación (Sección 3.2.1 RF-09, 3.6)
Sección 5.2 — Tiempo de respuesta (p50/p90/p99), throughput, SUS score, disponibilidad	No medido	Métricas de aplicación web exigidas por el formato; no se ha ejecutado una prueba de carga ni un cuestionario SUS a la fecha de este informe (Sección 5.2)